

Strategy Conquest

1. Úvod

1.1 Cíl projektu

Úkolem této semestrální práce je navrhnout testy a otestovat naši desktopovou herní aplikaci pomocí automatizovaných a manuálních testů. Testování bude zaměřeno na ověření hlavních funkcionalit herního enginu, validace dat a síťové komunikace pomocí frameworků JUnit a Mockito. Výsledkem práce je úspěšné absolvování předmětu Testování softwaru.

1.2 Cíl dokumentu

Cílem tohoto dokumentu je popsat testovanou aplikaci a navrhnout testovací strategii včetně testovacích scénářů. Dokument vychází z principů testování probíraných během semestru. Součástí práce je také implementace vybraných testů pomocí automatizace.

1.3 Autoři

Autory této semestrální práce jsou dva studenti: Vasyl Klius a Ivan Hryshyn. Práce byla vypracována samostatně bez účasti dalších členů týmu. Autoři si jsou vědomi možných nedostatků a nepřesností, které mohou vzniknout během práce.

1.4 Výběr aplikace

Pro testování jsme si vybrali naši desktopovou aplikaci Strategy Conquest. Jedná se o tahovou strategickou hru naprogramovanou v jazyce Java, která nabízí lokální hru proti umělé inteligenci i síťový multiplayer. Aplikace umožňuje hráčům budovat ekonomiku, spravovat armádu, vylepšovat města a bojovat o absolutní kontrolu nad mapou.

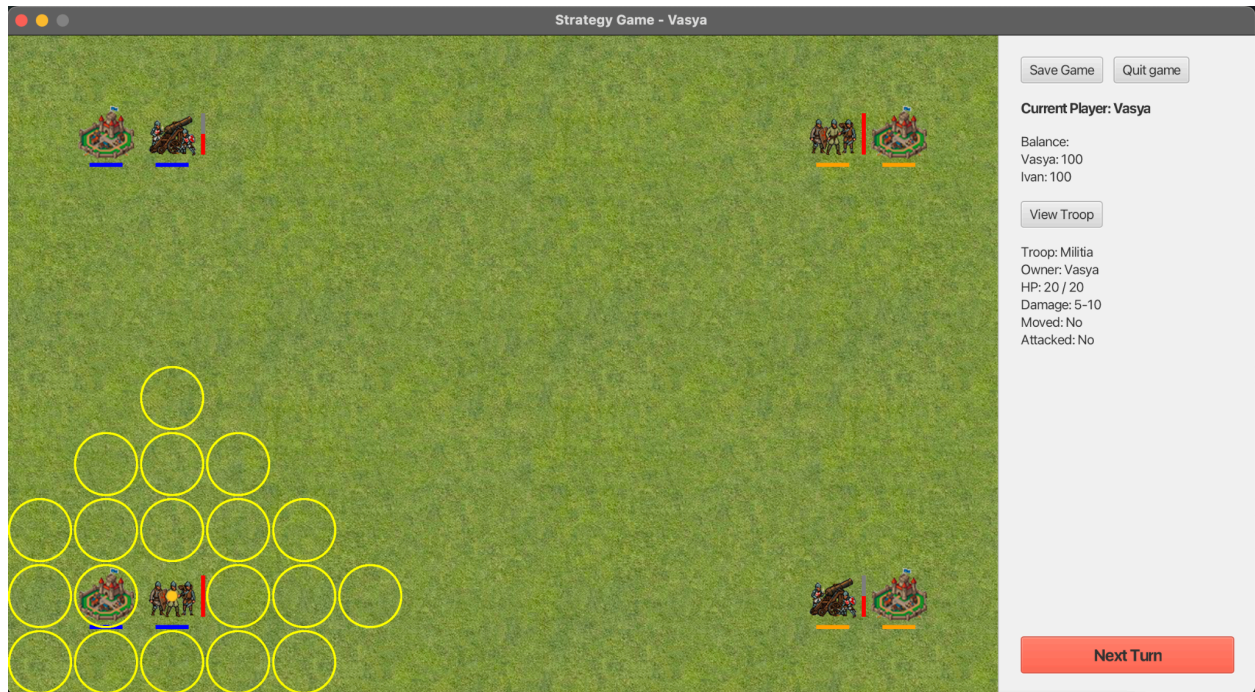
Hlavním důvodem výběru této aplikace je fakt, že se jedná o náš vlastní projekt z předchozího semestru. Chtěli jsme provést jeho důkladnou analýzu a profesionálně otestovat jeho robustnost, logiku a síťovou architekturu. Testování bude zaměřeno na simulaci chování běžného hráče pomocí manuálních E2E testů, a především na detailní prověření aplikačního jádra a síťových procesů pomocí automatizovaných testů.

2. Popis aplikace

2.1. Účel a hlavní funkcionalita aplikace

Název aplikace: Strategy Conquest

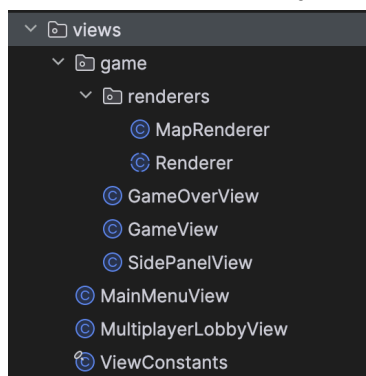
Aplikace je tahová strategická hra postavená na 2D mřížce (grid-based map), jejímž cílem je ekonomický rozvoj, správa armády a kompletní dominování nad protihráčem. Hráči začínají hru s hlavním městem a základní vojenskou jednotkou, přičemž každé kolo těží suroviny (zlato), které slouží pro nákup nových jednotek a vylepšování městských struktur. Hra končí v momentě, kdy jeden z hráčů ztratí kontrolu nad všemi svými městy.



2.2. Architektura a použité technologie

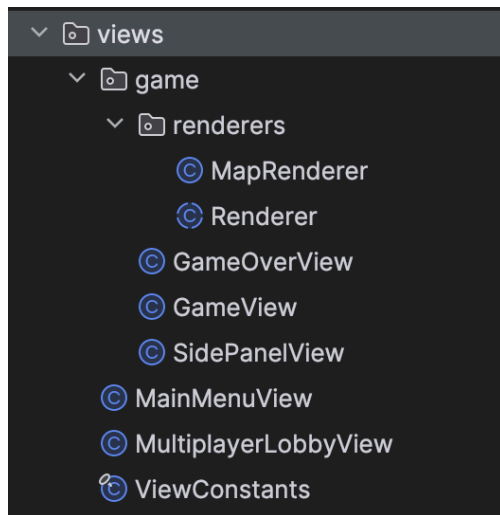
Aplikace striktně dodržuje návrhový vzor **Model-View-Controller (MVC)** a je implementována v jazyce Java. Z hlediska testování je architektura rozdělena do následujících klíčových vrstev:

- **Uživatelské rozhraní (Frontend):** Zajištěno frameworkem JavaFX (např. třídy `GameView`, `MainMenuView`). Samotná mapa je vykreslována pomocí vrstvených pláten (`Canvas`), které oddělují statické textury od interaktivních indikátorů tahů a útoků.

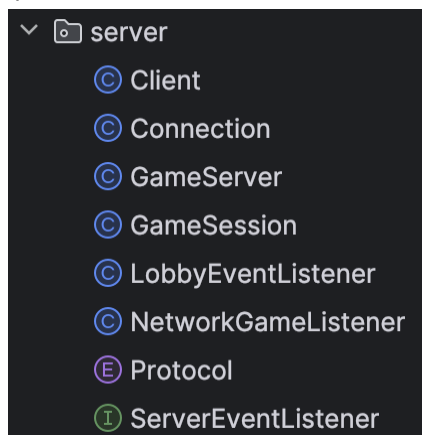


- **Aplikační a doménová logika (Backend):** Stav hry a její pravidla jsou zapouzdřeny v

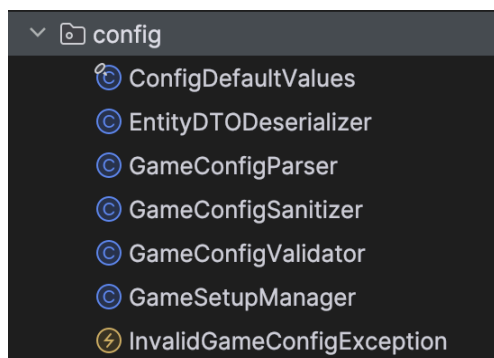
doménových modelech a spravovány dedikovanými manažery (např. omezení pohybu, výpočty poškození). Logika je izolována za rozhraním herních služeb (Service Layer).



- **Síťová komunikace:** Využívá se vlastní klient-server architektura postavená na TCP vrstvě (Socket). Komunikace mezi klienty a serverem probíhá asynchronně přes dedikovaná vlákna za využití vlastního textového protokolu oddělovaného znakem „pipe“ (|).



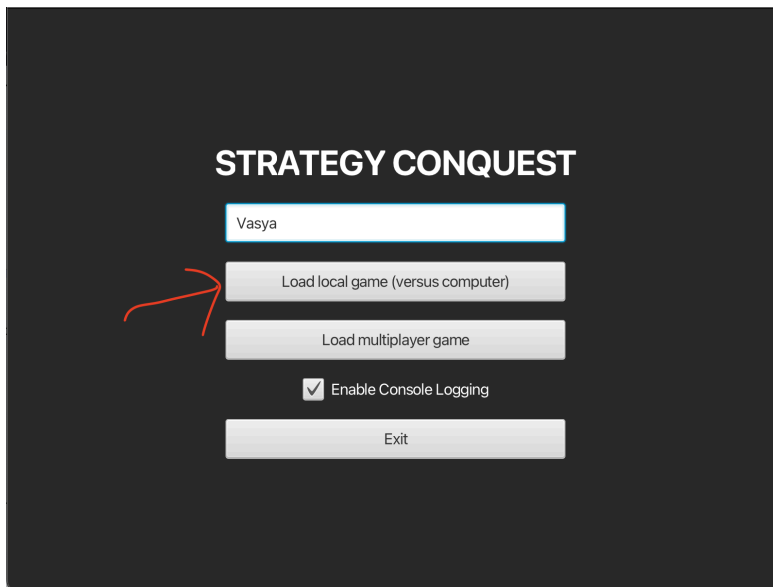
- **Ukládání a načítání dat:** Počáteční stavy a uložené pozice jsou spravovány pomocí serializace do formátu JSON (knihovna Gson). Systém obsahuje dedikované vrstvy pro sanitizaci a přísnou strukturální validaci načítaných konfigurací.



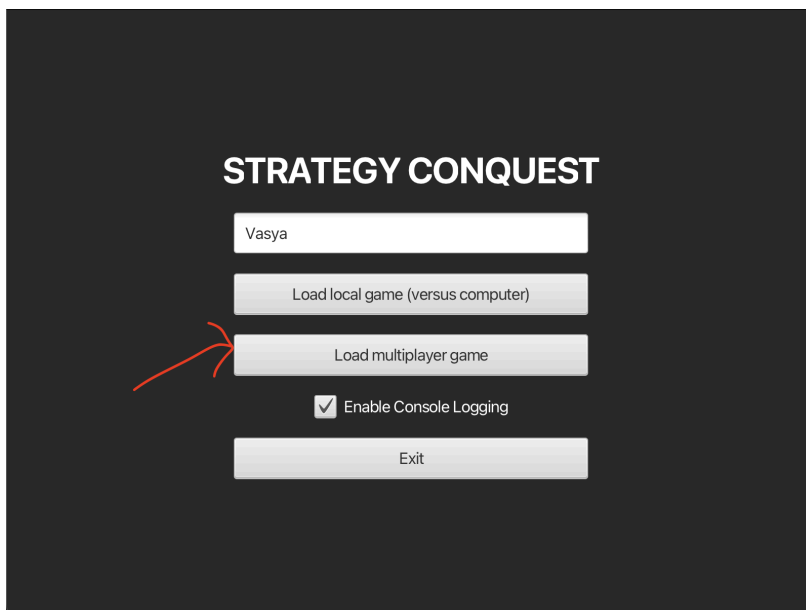
2.3. Provozní režimy (Cíle testování chování)

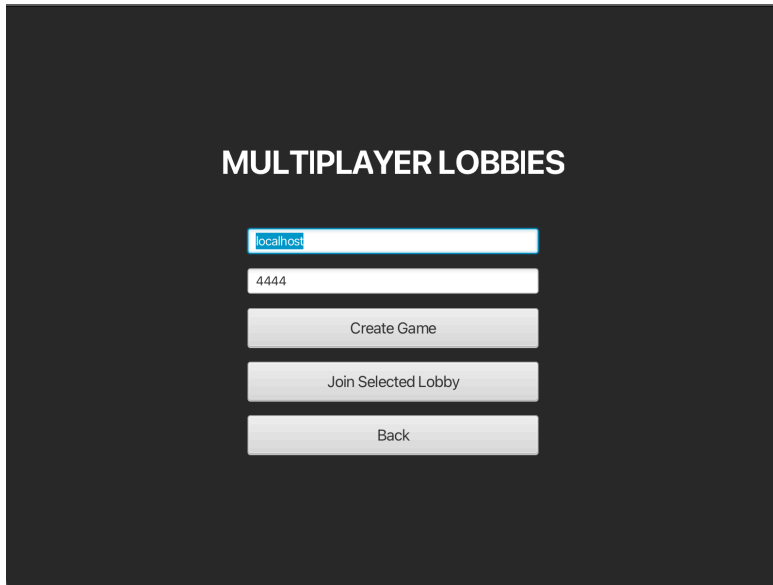
Aplikace podporuje dva hlavní režimy běhu, které vyžadují odlišné testovací přístupy:

- **Lokální hra (Single-player):** Hráč hraje proti lokální umělé inteligenci. Tahy oponenta jsou řízeny asynchronně na pozadí pomocí třídy BotExecutor, která vyhodnocuje heuristiku pro útok, pohyb a ekonomická rozhodnutí (nákupy a upgrady).



- **Síťová hra (Multiplayer):** Komunikace typu peer-to-peer (hostitel-klient) přes síť. Zahrnuje zakládání herních sezení (GameSession), ověřování klientů, synchronizaci herního stavu a bezpečné sdílení autoritativních tahů vyhodnocovaných serverem.





2.4. Kritické moduly pro návrh testů

Na základě specifikace lze identifikovat následující oblasti s nejvyšším rizikem, na které se testovací strategie musí zaměřit:

1. **Validace konfigurací (Data-driven testy):** Odolnost systému proti nevalidním formátům JSON souborů, chybějícím hráčům, kolizím souřadnic a neplatným herním atributům.
2. **Herní mechaniky (Unit testy):** Přesnost výpočtů v boji, validace pathfindingu pro korektní pohyb jednotek po mřížce a vynucování stavů tahů (jednotka nesmí táhnout dvakrát).
3. **Síťová robustnost (Integrační testy):** Zacházení s neočekávaným odpojením klienta, pády socketů během probíhající hry a prevence desynchronizace mapy mezi dvěma klienty.
4. **Asynchronní stabilita AI (Behaviorální testy):** Ověření, že výpočty bota na pozadí nezpůsobí zamrznutí UI vlákna nebo "softlock", pokud hra nastartuje přímo tahem bota.

3. Testovací strategie

3.1. Přehled částí aplikace

Aplikaci lze z hlediska testovatelnosti logicky rozdělit do 5 hlavních modulů:

Modul Herní logiky a pravidel (Core Engine)

Toto je nejdůležitější a nejcitlivější část aplikace (tzv. business logika). Definuje, co se ve hře

smí a nesmí stát. Je ideálním kandidátem na přísné automatizované **Unit testy**.

- **Správa pohybu (MovementManager):** Validace dosažitelnosti políček, blokování pohybu přes překážky a jiné jednotky.
- **Bojový systém (CombatManager):** Výpočty zranění (včetně minimálních a maximálních hranic), odstraňování mrtvých jednotek z mřížky a léčení ve městech.
- **Ekonomika a správa (EconomyManager):** Kontrola dostatečného zůstatku zlata (balance) při nákupu jednotek a vylepšování (upgrade) měst.
- **Řízení tahů a vítězství (TurnManager, ConquestManager):** Předávání tahů mezi hráči, převod vlastnictví měst (transferCity) a detekce konce hry (vítěz/poražený).

Modul Správy dat a konfigurace (Data & Setup)

Tento modul se stará o to, jak se hra načítá, ukládá a jak se ověřuje validita dat z externích souborů (JSON). Vhodné pro **Data-driven testy** (testování různých vstupních JSON souborů).

- **Parsery a Deserializace:** Překlad JSON souborů do DTO (Data Transfer Objects) struktury.
- **Validátory (GameConfigValidator, Sanitizer):** Zajištění, že mapa nemá záporné rozměry, hráči nemají duplicitní jména, jednotky se nepřekrývají a parametry neobsahují nepovolené znaky.
- **Setup a State Management (GameSetupManager):** Sestavení finálních doménových modelů (Game, Player, Map) z ověřených DTO.

Modul Síťové komunikace (Networking Layer)

Část zajišťující synchronizaci mezi klienty v multiplayeru. Silný kandidát na **Integrační testy** a testy ošetření chyb (Error handling).

- **GameServer a GameSession:** Zakládání místností, ověřování klientů (matching podle configu) a autoritativní zpracování požadavků od klientů.
- **Client a Connection:** Čtecí vlákna pro příjem a dekodování zpráv, serializace požadavků na formát vlastního protokolu (vázané pipe znakem |).
- **Službová abstrakce (Service Layer):** Překlad uživatelských akcí do sítě přes **NetworkGameService** a čekání na odpověď serveru.

Modul Umělé inteligence (AI Bot)

Tento modul simuluje lidského hráče. Z hlediska testování jde o ověření chování (Behaviorální testy).

- **BotExecutor:** Skript, který vyhodnocuje herní plochu, hledá nejbližší cíle (heuristika), nakupuje jednotky podle zůstatku a asynchronně provádí tahy na pozadí.

Uživatelské rozhraní a Zobrazení (Frontend / UI)

Grafická vrstva postavená na JavaFX, která pouze zobrazuje stav z backendu a sbírá kliknutí uživatele. Zde se obvykle plánují **Manuální exploratorní testy** nebo **E2E (End-to-End) UI automatizace** (např. pomocí TestFX).

- **Vykreslování mřížky (MapRenderer, GameView):** Správné překreslení Canvasu, indikátory pohybu, tahů a útoků.
- **Interaktivní prvky:** Postranní panel (SidePanelView), nákupní menu, přepínání cílů.
- **Řízení oken (SceneDirector):** Plynulé přechody mezi hlavním menu, lobby a herní obrazovkou. Možnost záchrany stavu při zavření okna.

3.2.Prioritizace částí aplikace

1. Priorita: Herní logika a pravidla (Core Engine)

- **Přístup:** Striktní automatizované Unit testy.
- **Zdůvodnění:** Absolutní priorita. Moduly jako MovementManager, CombatManager nebo EconomyManager tvoří jádro aplikace. Pokud selže ověřování pohybu, nebo se špatně spočítá poškození, celá hra ztrácí integritu. Chyby zde mají fatální dopad na funkčnost. Tato vrstva je navržena tak, že nemá žádné závislosti na uživatelském rozhraní nebo síti, takže se testuje nejlépe, nejrychleji a nejlevněji. V této části je nutné požadovat nejvyšší možné pokrytí kódu (Code Coverage).

2. Priorita: Správa dat a konfigurace (Data & Setup)

- **Přístup:** Datově řízené Unit testy a Integrační testy.
- **Zdůvodnění:** Systém spoléhá na deserializaci JSON souborů pro načítání herních stavů. Pokud třída GameConfigValidator propustí nevalidní stav (např. překrývající se jednotky, chybějící IDs, záporné životy), aplikace havaruje nebo se dostane do neopravitelného stavu. Vzhledem k tomu, že DTO objekty se serializují a odesílají i po síti, funguje tato validační vrstva jako kritický bezpečnostní a stabilizační prvek.

3. Priorita: Síťová komunikace (Networking Layer)

- **Přístup:** Integrační testy s mockováním síťových rozhraní.
- **Zdůvodnění:** Síťový kód je inherentně náchylný k chybám (deadlocky, ztráta spojení, desynchronizace stavu mapy mezi klienty). Selhání v této vrstvě způsobí pád multiplayerových relací. Testování je zde ale technicky složitější, protože vyžaduje práci s více vlákny a TCP sokety. Prioritou je otestovat robustnost vlastního komunikačního protokolu a reakce systému na nestandardní stavy (např. náhlé odpojení oponenta).

4. Priorita: Umělá inteligence (AI Bot)

- **Přístup:** Behaviorální testy.
- **Zdůvodnění:** Střední až nižší priorita. Pokud BotExecutor vybere suboptimální tah z hlediska herní strategie, je to logická vada, ale neblokuje to běh celé aplikace (nezpůsobí pád). Kritické je zde otestovat pouze to, že asynchronní spuštění bota na pozadí nezablokuje hlavní vlákno (zamezení softlocku) a že se bot nepokusí provést tah, který porušuje pravidla enginu.

5. Priorita: Uživatelské rozhraní (Frontend / UI)

- **Přístup:** Výhradně manuální exploratorní testování.
- **Zdůvodnění:** Nejnižší priorita pro testování, obzvláště pro automatizaci. Psát E2E testy pro překreslování JavaFX Canvas komponent je z hlediska poměru ceny a výkonu neefektivní. Testy by byly křehké a při každé změně designu by selhaly. Pokud je backend (Core Engine) robustní a stabilní, UI chyby jsou povětšinou pouze vizuálního charakteru (špatně vykreslený polygon, překrývající se text). Tuto část je nutné z automatizace vyřadit a spolehnout se na manuální průchod.

3.3. Test levels

1. Testování komponent (Component / Unit Testing)

Cíl: Nejnižší úroveň testování, která izolovaně ověřuje správnost jednotlivých tříd, metod a algoritmů bez závislostí na databázi, souborovém systému nebo uživatelském rozhraní.

Klíčové oblasti v projektu:

- **Herní manažeři (Core Logic):** Ověření izolovaných funkcí. Například výpočet dostupných políček pro pohyb v MovementManager, výpočet poškození a léčení v CombatManager nebo kontrola dostatku zlata pro nákup jednotek v EconomyManager.
- **Validace dat:** Testování hraničních hodnot a neplatných vstupů ve třídě GameConfigValidator. Musí být otestováno, že validátor správně vyhodí výjimku při duplicitních jménech hráčů, záporných rozměrech mapy nebo při kolizi jednotek na jednom políčku.
- **Technologie:** JUnit 5 pro běh testů, Mockito pro nahrazení závislostí (např. mockování objektu Player při testování logiky TroopUnit).

2. Integrační testování (Integration Testing)

Cíl: Ověření komunikace a datových toků mezi dvěma a více moduly, které byly samostatně otestovány v předchozí fázi. Rozděluje se na integraci komponent a integraci subsystémů.

Klíčové oblasti v projektu:

- **Integrace komponent (Setup Pipeline):** Tok dat při načítání hry. Test musí ověřit, že GameConfigParser správně přečte JSON soubor, předá data GameConfigValidatoru k ověření, a následně GameSetupManager úspěšně zkonstruuje instance doménových objektů (např. GameMap, City, BotPlayer). Zde se pracuje se skutečnými soubory namísto mocků.
- **Systémová integrace (Síťová vrstva):** Komunikace mezi herním serverem a klientem přes TCP sokety. Testuje se, zda akce vyvolaná na klientovi (např. metoda moveUnit ve třídě Client) správně zformátuje textový protokol (např. MOVE|unitId|x|y), zda zprávu Connection na straně serveru správně dekoduje a zda GameSession zpětně rozešle aktualizovaný stav (broadcast).

3. Systémové testování (System Testing)

Cíl: Testování aplikace jako jednoho kompletního a propojeného celku. Ověřuje se, zda systém splňuje technické a funkční specifikace. Testuje se přes definovaná rozhraní (např. API nebo UI) v prostředí, které simuluje produkci.

Klíčové oblasti v projektu:

- **End-to-End (E2E) herní scénáře:** Spuštění celého životního cyklu lokální hry. Vytvoření mapy, odehrání tahu lidským hráčem, předání tahu, spuštění asynchronního vlákna pro tah BotExecutor a ověření, že po dokončení útoku a pohybu bota se tah korektně vrátí zpět lidskému hráči.
- **Stabilita a zotavení (Non-functional testing):** Systémové testy multiplayeru. Co se stane, když se jeden z klientů neočekávaně odpojí (násilné ukončení soketu). Server musí tuto situaci zachytit a ukončit hru pro zbývajících hráče zprávou GAME_OVER.
- **Testování asynchronních operací:** Ověření, že běh AI algoritmu v CompletableFuture blokuje herní plochu před neoprávněnými zásahy uživatele (zabránění "race conditions"), ale zároveň nezpůsobuje zásek (zamrznutí) hlavního vykreslovacího vlákna JavaFX.

4. Akceptační testování (Acceptance Testing)

Cíl: Formální ověření, že aplikace splňuje byznysové a uživatelské požadavky ze zadání semestrální práce (UAT - User Acceptance Testing). Testování se provádí z pohledu koncového uživatele, zpravidla formou manuálního průchodu (black-box).

Klíčové oblasti v projektu:

- **Validace pravidel hry (Alpha testování):** Manuální hraní za účelem zjištění, zda je herní mechanika vyvážená a odpovídá specifikaci. Ověření, že systém správně detekuje konec hry a vyhlásí vítěze ve chvíli, kdy protihráč přijde o své poslední město.
- **Uživatelské rozhraní (Usability):** Intuitivnost ovládání v UI JavaFX. Zda se správně překreslují interaktivní vizuální prvky (zelené kruhy pro pohyb, červené pro útok), zda je zřejmé, čí tah právě probíhá, a zda chybové hlášky v lobby (např. při snaze připojit se do

plné hry) dávají uživateli smysl.

4. Testy vstupů

Pro testování byly vybrány dva netriviální vstupy aplikace Strategy Conquest:

- zpracování síťového příkazu pohybu (Connection.handleMove)
- validace nákupu vojenské jednotky (GameSession.onUnitPurchase)

4.1 Ekvivalenční třídy a mezní podmínky

4.1.1 Vstup 1 – Síťový příkaz pohybu

Tento vstup přijímá textový paket rozdělený na pole řetězců (např. ["MOVE", "u1", "5", "6"]). Analyzujeme délku pole a formát souřadnicových parametrů.

EC ID	Popis	Příklad
EC1	Validní formát a délka pole (4+ parametry)	["MOVE", "u1", "5", "6"]
EC2	Nevalidní délka pole (chybějící data)	["MOVE", "u1", "5"]
EC3	Číselná souřadnice v rámci mapy (např. 10x10)	"5"
EC4	Nečíselný znak v souřadnici (písmena, symboly)	"PĚT", "@"
EC5	Souřadnice mimo hranice mapy	"-1", "15"

ID	Podmínka	Hodnota	Očekávané chování
B1	Minimální platná délka pole	4 parametry	Přesun se provede
B2	Hraniční neplatná délka pole	3 parametry	Chyba (MOVE_REQUIRES_UNITID_X_Y)
B3	Minimální platná souřadnice	0	Přesun se provede
B4	Maximální platná souřadnice	9	Přesun se provede
B5	Překročení rozměru mapy	10	Omezení / chyba (INVALID_COORDINATES)

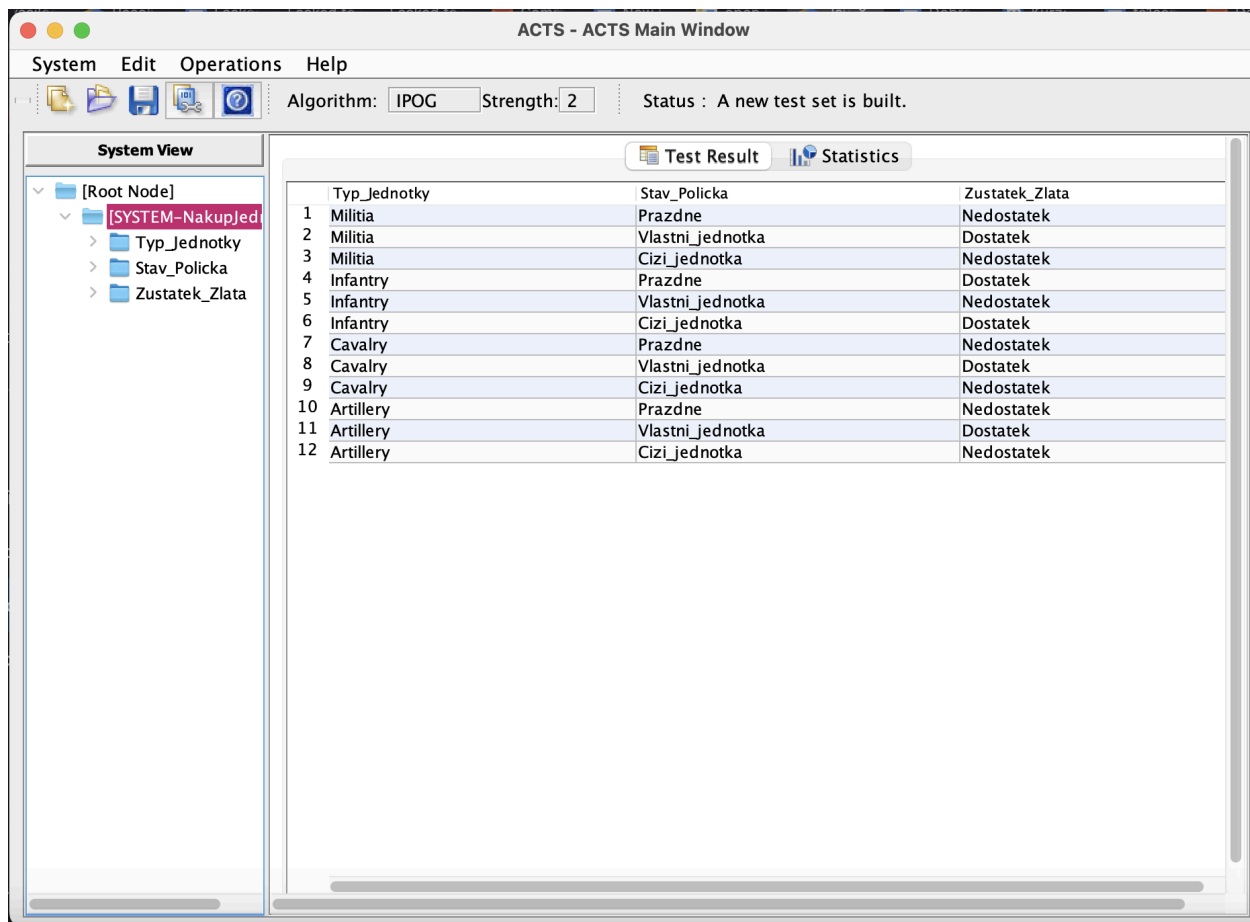
4.1.2 Vstup 2 – Nákup vojenské jednotky

Tento vstup analyzuje validaci textového výčtového typu (Enum) a byznysovou podmínku zůstatku zlata hráče (pro test předpokládáme cenu jednotky Militia = 50 zlata).

EC ID	Popis	Příklad
EC1	Platný typ jednotky (shoda s Enumem)	Militia, Infantry
EC2	Neplatný nebo neexistující typ jednotky	Tank
EC3	Typografická chyba v typu jednotky	militia, Militia
EC4	Dostatečný zůstatek zlata	100 zlata
EC5	Nedostatečný zůstatek zlata	20 zlata

ID	Podmínka	Hodnota	Očekávané chování
B1	Dostatek zlata - hraniční minimum	50 zlata	Nákup úspěšný, zůstatek 0
B2	Nedostatek zlata - hraniční maximum	49 zlata	Chyba (ERROR_BUYING_TROOP)
B3	Běžný dostatek zlata	100 zlata	Nákup úspěšný, zůstatek 50
B4	Extrémní záporný zůstatek	-1 zlato	Chyba (ERROR_BUYING_TROOP)

4.2 Pairwise testing



Kombinace vstupních dat byly vytvořeny pomocí nástroje ACTS.

Vzhledem k tomu, že byly použity tři parametry s více hodnotami (Typ jednotky [4], Stav políčka [3] a Zůstatek zlata [2]), vyčerpávající testování (exhaustive testing) by znamenalo testování 24 možných kombinací ($4 \times 3 \times 2 = 24$).

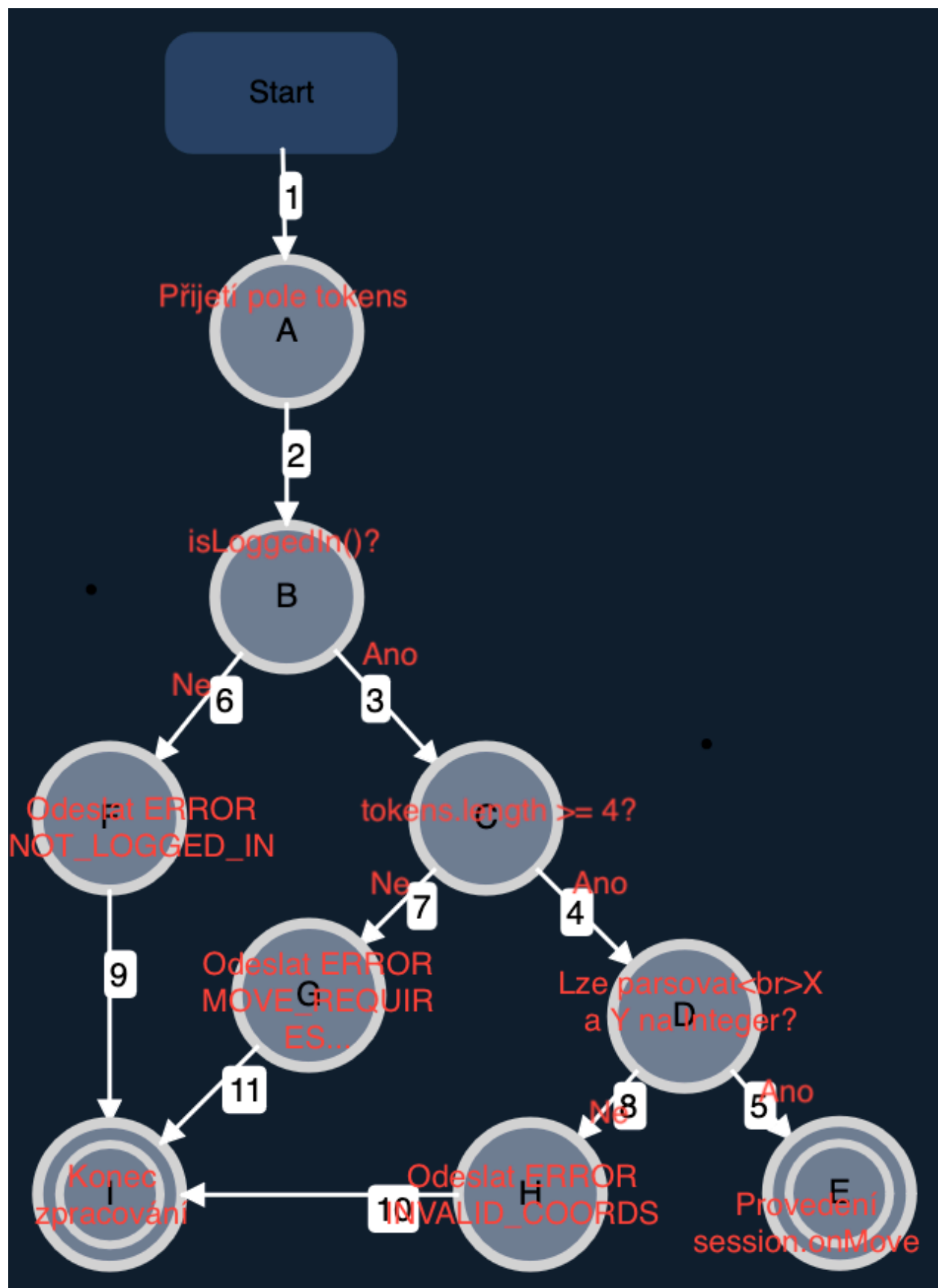
Použití techniky pairwise testing vedlo k redukci na **12 testovacích scénářů**, které plně pokrývají interakce všech dvojic parametrů. V tomto případě tedy došlo ke snížení počtu testovacích scénářů o 50 %, což výrazně šetří čas nutný pro testování při zachování vysoké úrovně pokrytí integrace parametrů.

5. Testy průchodu (Process Testing)

5.1. Proces 1: Zpracování síťového příkazu pohybu

Tento proces modeluje běh metody handleMove ve třídě Connection, která přijímá příkaz k přesunu od klienta, validuje stav vlákna, kontroluje počet parametrů a následně se je pokouší převést na souřadnice.

5.1.1. Diagram řízení toku (Control Flow Graph)



Seznam uzlů:

- **1:** Vstupní bod (metoda přijala parametry).
- **2, 3, 4:** Rozhodovací uzly (Decision nodes) validující postupně autorizaci, kompletnost dat a typovou integritu.
- **5, 6, 7, 8:** Uzly akcí (Action nodes) realizující provedení tahu, nebo odeslání konkrétní chybové zprávy.
- **9:** Koncový uzel.

5.1.2. Návrh testovacích cest pro TDL 2 (Pokrytí hran)

Abychom dosáhli úrovně TDL 2, musíme projít všechny hrany (šípky) v grafu. Navrhujeme 4 nezávislé testovací scénáře, které pokrývají 100% hran.

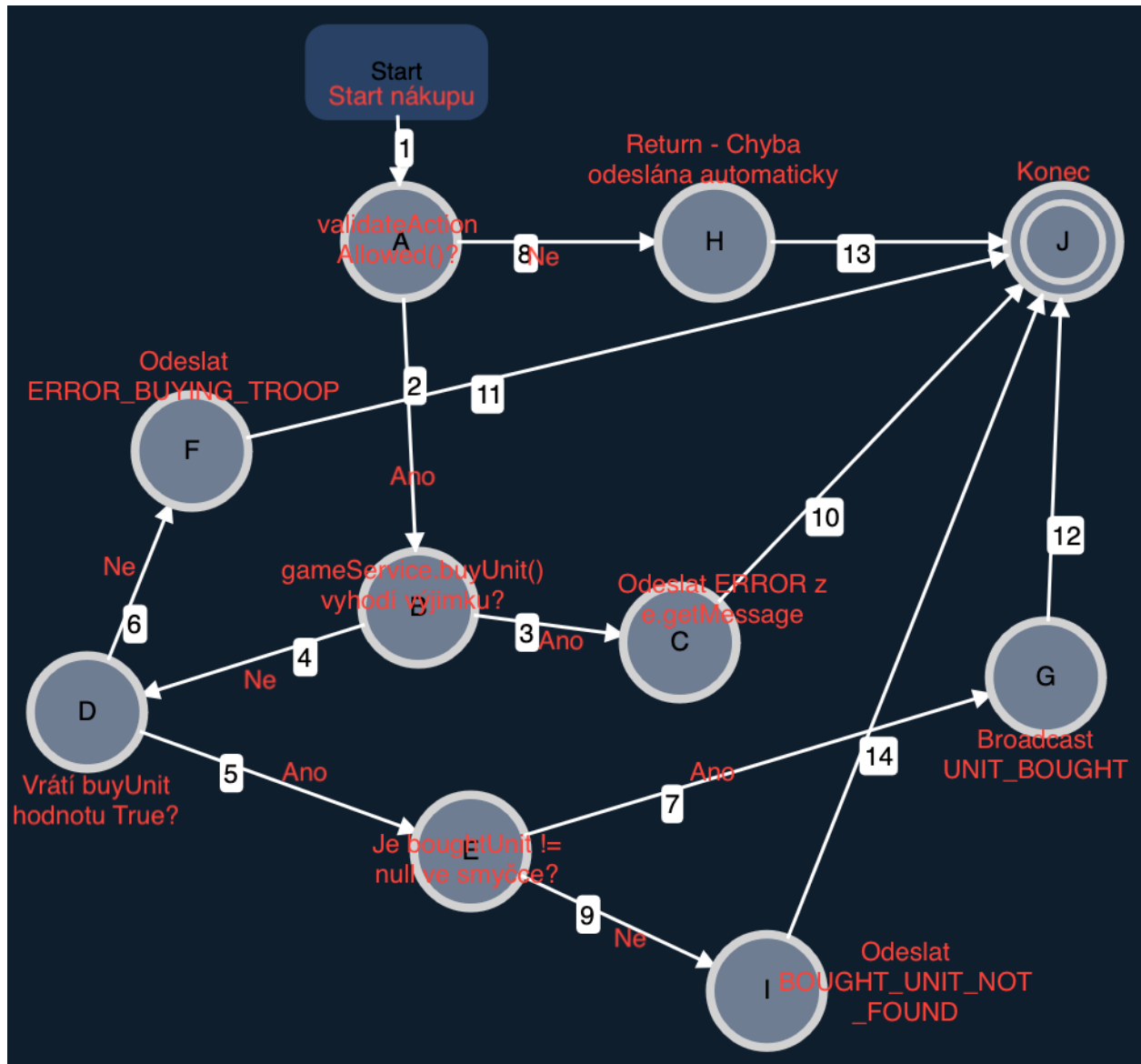
ID Cesty	Procházené uzly (Cesta)	Vstupní data / Předpoklad	Očekávaný výsledek	Pokryté hrany (Nové)
P1-01	1 --> 2 --> 3 --> 4 --> 5 --> 9	Klient je přihlášen, délka je 4, data "5", "6".	Přesun se úspěšně provede.	1-->2(Ano), 2-->3(Ano), 3-->4(Ano), 4-->5
P1-02	1 --> 2 --> 6 --> 9	Klient není přihlášen.	Odesláno NOT_LOGGED_IN.	2-->6(Ne)
P1-03	1 --> 2 --> 3 --> 7 --> 9	Klient je přihlášen, délka je pouze 3 .	Odesláno MOVE_REQUIRES.. ..	3-->7(Ne)
P1-04	1 --> 2 --> 3 --> 4 --> 8 --> 9	Přihlášen, délka 4, ale data "PĚT", "6".	Odesláno INVALID_COORDS.	4-->8(Ne)

Vyhodnocení TDL 2: Všechna rozhodnutí (uzly 2, 3, 4) byla otestována na hodnotu True i False. Bylo dosaženo 100% pokrytí hran.

5.2. Proces 2: Zpracování nákupu jednotky (Multiplayer Session)

Tento proces mapuje logiku metody onUnitPurchase ve třídě GameSession. Proces zahrnuje bezpečnostní ochranu tahu, volání aplikační služby, ošetření výjimek a finální ověření vytvoření objektu před synchronizací všem klientům.

5.2.1. Diagram řízení toku (Control Flow Graph)



Seznam uzlů:

- 1: Zahájení procesu uvnitř session.
- 2, 3, 5, 6: Rozhodovací uzly zjišťující: oprávnění uživatele (zda je na tahu), robustnost engine (try-catch), stavový výsledek engine a existenci nového objektu.
- 4, 7, 8, 9, 10: Akční uzly pro modifikaci stavu komunikace a rozesílání zpráv.
- 11: Koncový uzel metody.

5.2.2. Návrh testovacích cest pro TDL 2 (Pokrytí hran)

Navrhujeme 5 nezávislých cest, abychom propálili všechny možné výstupy podmíněných větvení a bloků try-catch v tomto byznysovém procesu.

ID Cesty	Procházené uzly (Cesta)	Vstupní data / Předpoklad	Očekávaný výsledek
----------	-------------------------	---------------------------	--------------------

P2-01	1 --> 2 --> 3 --> 5 --> 6 --> 8 --> 11	(Happy Path): Hráč je na tahu, engine je zdravý, nákup vrátí True, jednotka je nalezena na políčku.	Broadcast úspěchu všem.
P2-02	1 --> 2 --> 9 --> 11	Požadavek odešle hráč, který není na tahu (nebo hra nezačala).	Brzké ukončení metody (v uzlu 2 je odeslána chyba NOT_YOUR_TURN).
P2-03	1 --> 2 --> 3 --> 4 --> 11	Nákup selže fatální výjimkou (např. neplatný Enum jednotky vyvolá IllegalArgumentException).	Odchycení výjimky, odeslání zprávy přes e.getMessage().
P2-04	1 --> 2 --> 3 --> 5 --> 7 --> 11	Hráč nemá peníze nebo je město blokováno. buyUnit() nehodí chybu, ale vrátí false.	Odesláno ERROR_BUYING_TROOP.
P2-05	1 --> 2 --> 3 --> 5 --> 6 --> 10 --> 11	Systém vrátil úspěch, ale novou jednotku se na mapě nepodařilo fyzicky dohledat (boughtUnit == null).	Odesláno BOUGHT_UNIT_NOT_FOUND.

Vyhodnocení TDL 2: Testovací cesty pokrývají kompletně všechny Yes/No výstupy z uzlů 2, 3, 5 i 6. Tímto přístupem je prokázána 100% integrace a připravenost na pokrytí hran pro kritický síťový proces zpracování tahů.

6. Detailní testovací scénáře (Detailed Test Cases)

V této sekci jsou rozpracovány dva end-to-end (E2E) testovací scénáře, které ověřují integraci uživatelského rozhraní (JavaFX) s jádrem herního engine a síťovou vrstvou. Scénáře jsou zapsány ve standardizovaném formátu vhodném pro manuální i automatizované testování.

6.1. Testovací scénář 1: Úspěšný nákup jednotky z

pohledu uživatele

Tento scénář ověřuje, že se správně překládá uživatelské kliknutí v grafickém rozhraní až do změny stavu herní ekonomiky a že se UI na základě této akce korektně překreslí.

Atribut	Hodnota
ID testu:	TC-E2E-001
Název testu:	Korektní nákup a vykreslení nové jednotky ve vlastním městě
Modul:	GUI / GameController / EconomyManager
Priorita:	Vysoká (High)
Předpoklady (Pre-conditions):	1. Aplikace běží a herní mapa je načtena. 2. Hráč "Player1" je aktuálně na tahu. 3. Hráč "Player1" vlastní město na souřadnicích X=1, Y=1. 4. Políčko X=1, Y=1 je prázdné (nestojí na něm voják). 5. Zůstatek zlata "Player1" je 100.
Testovací data:	Typ jednotky k nákupu = Militia (cena 50 zlata).

Postup testu (Test Steps):

Krok	Akce uživatele / Systému	Očekávaný dílčí výsledek
1.	Uživatel klikne levým tlačítkem myši na políčko (1,1) reprezentující jeho město.	Metoda handleGameAreaClick identifikuje kliknutí na entitu. Město je vybráno (okolo něj se objeví zvýraznění).
2.	Uživatel zkontroluje informační panel (SidePanelView).	Postranní panel zobrazuje detaily města a obsahuje tlačítka pro nákup jednotek (vlastnost isOwner = true).
3.	Uživatel klikne na tlačítko "Koupit Militia".	Akce vyvolá handleBuyUnit(cityId, "Militia"). Systém odešle požadavek do aplikační služby.
4.	Systém ověří byznysová pravidla na pozadí a potvrdí nákup.	Zůstatek zlata hráče je snížen ze 100 na 50.
5.	Ovladač (Controller) zavolá	Na plátně (Canvas) se na

	metodu updateTile(x, y) pro překreslení políčka.	políčku (1,1) vizuálně objeví nová ikona (Sprite) jednotky Militia patřící hráči "Player1".
--	--	---

- **Očekávaný výsledek (Post-condition):** Nákup proběhl úspěšně. Účet hráče je zatížen částkou 50 zlata. Nová jednotka je evidována v globálním stavu GameDTO a je vizuálně přítomna na herní ploše.

6.2. Testovací scénář 2: Kapitulace a ukončení hry

Tento scénář ověřuje proces, kdy jeden z hráčů vzdá hru. Testuje se schopnost systému detekovat tuto událost na klientské vrstvě, určit vítěze a správně delegovat zobrazení obrazovky "Game Over" do hlavního vlákna uživatelského rozhraní.

Atribut	Hodnota
ID testu:	TC-E2E-002
Název testu:	Ukončení hry (Quit) a korektní vyhlášení vítěze
Modul:	GameController / AbstractClientService / SceneDirector
Priorita:	Střední (Medium)
Předpoklady (Pre-conditions):	1. Hra je v aktivním běhu. 2. Účastní se hráči "Player1" a "Player2". 3. Je aktivní tah hráče "Player1".

Postup testu (Test Steps):

Krok	Akce uživatele / Systému	Očekávaný dílčí výsledek
1.	Uživatel ("Player1") klikne na tlačítko "Odejít ze hry" (Quit Game).	Je spuštěna metoda handleQuitGameRequest() v kontroleru.
2.	Kontroler vyvolá metodu quit() na vrstvě herní služby (gameService.quit()).	Služba najde ve hře druhého hráče ("Player2") a označí ho za vítěze.
3.	ConquestManager oznámí výhru a odpálí událost přes naslouchače (onGameOver.accept(winnerName)).	Událost dorazí do GameControlleru.
4.	GameController zachytí událost a zavolá Platform.runLater() pro bezpečnou modifikaci UI.	Požadavek na změnu obrazovky je předán do vlákna JavaFX.

5.	Spustí se metoda sceneDirector.showGameOver Popup(winnerName).	Vyskakovací okno se zobrazí přes herní plochu s textem indikujícím výhru hráče "Player2".
----	--	--

- **Očekávaný výsledek (Post-condition):** Herní relace je ukončena. Oponent je automaticky prohlášen za vítěze na základě kapitulace prvního hráče. Vizuální zobrazení hry je překryto oznámením o konci hry. Hra nadále nereaguje na klikání do mřížky.

7. Implementace testů